

Root Cause Analysis in 5G/6G Networks

Dinis Canastro*, Ricardo Rocha*, Mário Antunes†, Diogo Gomes†, Rui L. Aguiar†

* Nokia/Universidade de Aveiro (e-mail: {dinis.canastro.ext, ricardo.rocha}@nokia.com)

† Instituto de Telecomunicações/Universidade de Aveiro (e-mail: {mario.antunes, dgomes, ruilaa}@ua.pt)

Abstract—Network Softwarization, the process by which network equipment's are being replaced by software running in a cloud environment, is playing an important role in the transformation of next-generation networks starting with 5G. In this environment, the use of AI/ML aided network automation to perform management tasks is vital to provide a more reliable and cost-effective network. An aspect in which AI/ML can play an important role is on the quick diagnose and track of root causes of anomalies. In this paper, we propose a system capable of automating root cause analysis in a cellular network scenario. By collecting log files from negative-impacting events occurring throughout the whole network levels, we are able to find the correlation between them according to a series of defined rules and track down the root cause using a graph-based approach to dependencies.

Index Terms—Computer Network Management, Root Cause Analysis, 5G Slicing

I. INTRODUCTION

The success of 5G depends, amongst other aspects, on the level of automation that can be achieved on the Network and Service Management layers. Automation plays a crucial role in minimising TCO (total cost of ownership) and is vital to fully leverage and capitalise the benefits introduced by 5G, especially 5G Slicing. 5G Slices can be very dynamic in terms of requirements, placement, and life cycle management and will create an active network usage to address ephemeral services [1]. Due to the complexity introduced by underlay and overlay layers, the management entities require automated services and procedures to manage the 5G networks' complexity.

The effective resource usage optimisation, Service-Level Agreement (SLA) fulfilment, and constant on-boarding and termination of slices, implies constant changes on all layers due to the lack of a unique, centralised, and holistic management entity. The dynamism observed is a consequence of the nature of the supported services. The different network domains (Access, Transport, Telco Cloud Infrastructure, etc.) are exposed to dynamic environments and requirements that will make the underlay not static. Additionally, the distributed (horizontal and vertical) aspect of the network represents a challenge for the Management Layer because of the data availability, observability, geographical area, technologies, and vendors involved.

Network and Service Management automation requires a closed-loop control in all the layers to ensure the management and technical purposes are met. ETSI Zero-touch network and Service Management (ZSM), ISG and 3GPP TSG Service and System Aspects WG5 Telecom Management (SA5) standardise the close-loop control to minimise the complexity

originated due to multi-vendor, multi-technology, and multi-domain aspects of the network [2].

Design (including on-boarding of functions and assets), planning, fulfilment, assurance, optimisation, and termination of services and slices demand effective and efficient automation using closed-loops control. All these phases need to be harmonised and compatible during all the life-cycle of the services [3]

The Assurance capabilities are vital to ensure service and network objectives are met, services require continuous monitoring and analytics to detect and identify current or potential issues. Closed-loop control systems depend on anomaly detection based on Thresholds Cross Analysis (TCA), Event or Alarm reception, policy engine, and predicted or inferred status (e.g., performance metrics). If an anomaly is detected, and before an action is taken, generically, a service to detect the cause of the anomaly is required, usually know as Root Cause Analysis (RCA) [4].

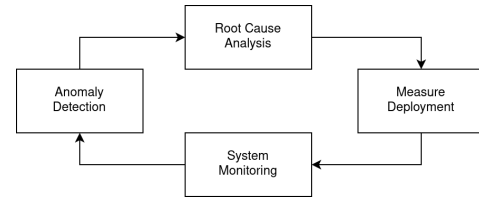


Fig. 1. Closed-Loop Flow

Considering the E2E Slice Latency Key Performance Indicator (KPI) degradation, the TCA system can trigger the Closed-Loop if 90% of the E2E Slice Latency KPI SLA is crossed. Before a corrective action is selected, an RCA service needs to first identify if the problem is in the overlay or in the underlay domain (Access, Transport, or Core), as shown in Figure 1. This flow is valid with any other identified Closed Loop trigger services, especially with Events or Alarms. Any simple drift or glitch in the underlay can be irrelevant in the domain but can severely impact any service or slice in the overlay.

As described, the ability to identify the Root Cause of a problem (or anomalies) is instrumental for the action selection and effectiveness of automation based on an Closed Loop control.

The remainder of the paper is organised as follows. In section II we detail the related work. Afterwards, we present a brief description of the system core workflow and requirements in the section III chapter followed by the section IV section

where we describe the process behind our system's validation and performance to measure the applicability in the real world of our concept. Finally, we present the section V featuring our final remarks on the development as well as future work we deem noteworthy.

II. RELATED WORK

RCA is a well known subject, but has not been thoroughly applied to 5G telecommunications networks. In [7], the solution proposed by the authors takes a decision aided by a dependency graph based on the network topology. By finding problems that are deemed correlated in multiple nodes sharing a dependency relationship, the system is capable of defining a chain of events and specify which one is the root cause. Similar concepts exist, some variants [5] do not use a dependency graph, rather opting to compare KPI's evolution to find correlation patterns among nodes. In [6], the authors relate the dependency graph together with the KPI's evolution to provide weights to the dependency graph so as to gather which nodes depend from others. All aforementioned articles so far have something in common though, they all use a knowledge base provided by a domain expert to evaluate the newly collected cases.

In others words surveys mention [8] *Assisted Generation* as a way to represent the models being worked on, but adding domain knowledge to them for better results. These are references to the type of system we intend to develop where we are looking for improved results by using an expert input as to how the system should behave in particular cases.

A review on multiple approaches [9] includes some solutions we deemed to be promising by using algorithms such as FP-Growth [10] or Random Tree Forest [11]. Algorithms capable of extracting rules that can pinpoint the root cause of anomalies. Those rules can then be transposed and applied to new events being processed in the RCA system.

We intend to grab multiple aspects of various works as well as experiment with new variants to implement a complete RCA workflow.

III. PROPOSED SOLUTION

In this paper, we propose an RCA solution that addresses the requirements of closed-loop control as identified by ETSI and 3GPP. We address the RCA problem in 5G/6G Networks through an incremental approach towards the ultimate goal of achieving a non-supervised RCA service capable of detecting, identifying, and proposing corrections to common 5G/6G network problems. The proposed solution encompasses four stages:

- 1) Retrieval of information from the network monitoring platform (event database).
- 2) Evaluation and grouping of events that forms chains of correlated events.
- 3) Retrieval of anomalies from a *knowledge base* and comparison with the previously obtained chains of events, returning a maximum similarity pair.

- 4) According to the maximum similarity value, a decision algorithm either accepts the correlation or declines it and adds it to a *temporary database*. Items left in the temporary database are chains we are not confident enough to provide a diagnose but should be re-evaluated through other means.

There are several paths one can take to solve each stage's main problem. We propose a few in our implementation as well as ensemble options where we use various in conjunction to improve the final results (refer to fig. 2).

A. Data Integration

The **Data Integration** phase will be responsible for the retrieval of each event from the *Event Database*. The collected events may encapsulate several fields, but we restrict to the ones used. The fields used are the data we consider imperative for a good evaluation while also being the data we consider that can be obtained from most FMS implementations for better portability. The fields used are:

- **Node** - Location of the event in the network.
- **Sample Timestamp** - The start timestamp when the alarm is being raised, and the end timestamp when being deactivated.
- **Error description** - Additional information such as the type of alarm and its parameters.
- **Status** - Specifies if it's an alarm being activated or deactivated.

B. Data Processing

This phase is composed by several steps. The first step, **Event Filtering**, will filter each event according to their branch. In parallel, we have the **Topology Update** stage, which is where we update our dependency graph. We can then proceed with the **Chain of Events Creation** where we start clustering events where we deem a correlation can be found and form chains of events. We present various alternative methods to form this chain of events all based on time windows [12].

1) *Event Overlay*: The first correlator implemented works by, firstly, filtering out events that happen in the same branch, which means they are only separated by the root of the graph according to the branch rules defined previously. Then we find which ones at any given time were both in an active state (briefly described in fig. 3). Meaning that if at some point in time, two events were active, belonging to the same branch, we assume they are correlated.

The way the comparison works is: let E_1 and E_2 be 2 different events in the same branch. ts_1 and ts_2 are the start timestamp of both E_1 and E_2 respectively while Te_1 and Te_2 are the end timestamp. If we compare $ts_1 \leq Te_2 \wedge Te_1 \geq ts_2$ and it's confirmed to be true, we group the 2 events on the same chain. We keep this comparison going until we reach a point where all the events are grouped, even if individually.

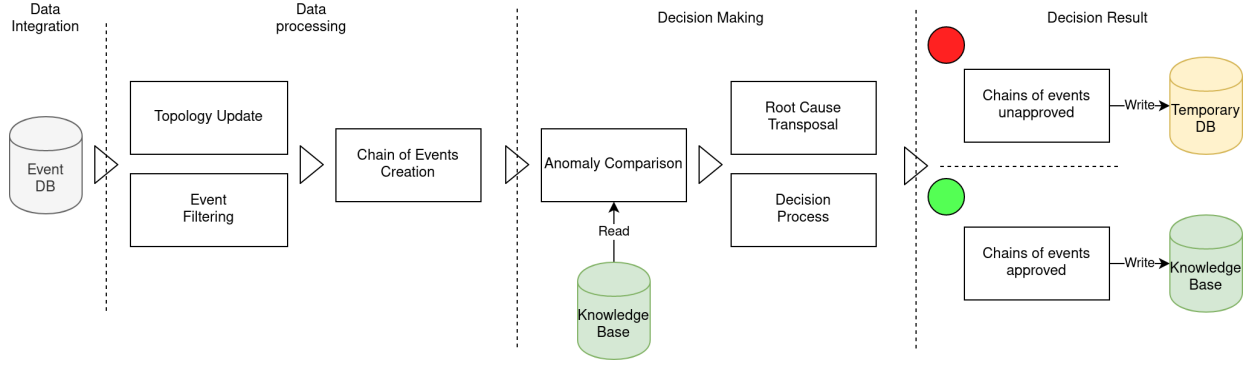


Fig. 2. Workflow Overview

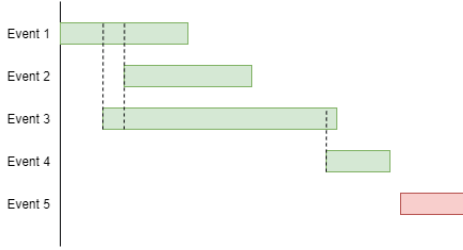


Fig. 3. Overlaying Correlation Example

2) *Static Window*: Our second correlation method is based on buckets with time windows. The time window works by comparing two event's starting timestamp. If they are less distant or equal to the defined window (described in fig. 4), we assume they are correlated and chain them together.

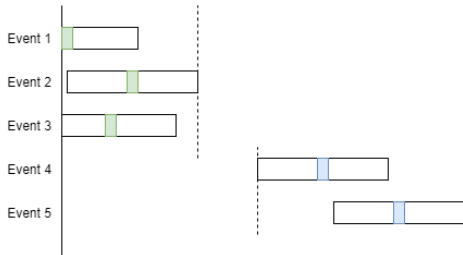


Fig. 4. Time Window Correlation Example

3) *Dynamic Window*: After collecting some initial results, we required a more sensible time window as the spacing between events varies depending on the anomaly. It's an evolution of the static time window method where we search the events from a specific branch and calculate the average window size for that specific branch on the current batch. It is worth noting this method works for systems with batches but would require a different way to calculate the window if we had a stream of events (e.g., using a number of the most recent events). The way the time window value is defined in this method is by calculating the average difference between

the current events under analysis.

$$W = \frac{\sum_{i=1}^{C_E} T_{i-1} - T_i}{C_E} * \frac{1}{R} \quad (1)$$

The equation reflects the window size W for a series of events E from a specific branch. The events must be ordered from the start timestamp T , and the number of events is C_E . R is a value greater or equal to 1, and it is intended to be used as a ratio to divide the average difference, which provided us with better results (can also be set to 1 if we intend not to use any ratio). This ratio is beneficial when we have scenarios with high deviation between event spacing and cases with small quantities of events from our evaluation. This dynamic window also allows us not to require to configure a specific time window for the correlator. The major downside is if the events included range from different time magnitudes that could compromise the results, such as having a different range from some seconds to multiple days. With the branching plus the batching, we reduce these cases' possibility to happen, yet it will depend on the system under analysis behaviour.

C. Decision Making

This stage is the most computational intensive. With the chains of events resulting from the last stage, we will pitch them against all the anomalies already confirmed and stored in the knowledge base. At this stage, each chain of events coming from the correlator will be called *candidate* while chains of events already approved and introduced in the knowledge base are called *anomalies*. There are many ways to compare graphs. We opted for a weighted average of a series of methods. This way, we are free to add, remove, or change a method's relevance in the comparison for different types of datasets. From this weighted average, we can obtain the most similar anomaly to the candidate chain of events and the similarity rate between both. In the end of the subsection we also explain the process behind obtaining a root cause suggestion.

1) *Hierarchy Comparison*: First and foremost we compare graphs by their connections according to their levels.

This method allows to compare layers or types of nodes without requiring possibly unavailable node information but instead calculate their distance to the root node. Similar node

types tend to share similar problems, hence the motivation behind this method. We then use intersection with duplicates (check eq. 2) to obtain the similarity value between 0 and 1.

$$\frac{L_1 \cap L_2}{\max(\text{len}(L_1), \text{len}(L_2))} \quad (2)$$

2) *Subgraph Comparison*: This method allows to, more precisely, compare topology and event spreading on the subgraph. We extract the origin node for each event belonging to the chains we are analyzing, and with those nodes, using the full topology graph, we build a complete graph which we call the chain of events' **subgraph**. If the events cannot build a complete subgraph, we search the shortest path to join the separated nodes into a unique, complete subgraph. We do this for both the candidate chain of events and the anomalies and calculate their Graph Edit Distance (GED). The GED algorithm returns the number of changes needed to transform one graph into the other.

This value is called the *GED*. We then require the number of nodes plus the number of edges in graph 1 and graph 2 which we call the graph size, N_1 and N_2 respectively. This information allows us to obtain the subgraph comparison value between 0 and 1 (using eq. 3). The graph 1 will always be the largest ones (the one containing the more number of nodes plus edges).

$$\text{Result} = \frac{N_1 - \text{GED}}{N_2} \quad (3)$$

3) *Average Branching Comparison*: The third method uses the subgraph extracted from both chains and calculates the average number of edges coming out of each node averaging by layer. Branching is a good pattern indicator in some scenarios as different types of nodes (identified by the layer in our case) exhibit similar amounts of connections.

We can compare this branching according to the absolute hierarchic position where the branching level is considered the shortest path distance to the root of the graph, or we have the relative hierarchical position where we consider the subgraph's highest level as the root. The first one is useful when each hierarchic level inherently explicit the type of nodes we are working with, just like our dataset, while the latter is better applied when we do not have such a rigid structure and the same layout may be found in different parts of the network.

4) *Error Description Comparison*: The used dataset includes information we consider relevant outside of the graph's mathematical properties such as error IDs, symptoms, or diagnostic information usually bundled in the events notification. This allows to differentiate better similar topological errors providing different outcomes according to the type of error (e.g., a link down or a port failure may exhibit the same anomaly layout yet require different repair procedures and should be labelled differently). This information is generally contained in strings, in our example, they shall be named S_1 and S_2 , and we compare them by using the

Levenshtein distance algorithm that counts the number of changes we would need to make S_1 equal to S_2 . This method is interesting because in our dataset, as in many others, the events usually follow a pattern of providing a general problem proceeded by the specifics (e.g., "Connection lost for IP=AL7.xQe.KMR.OYY"). So if the description follows that structure, we can find a degree of similarity between the same type of errors while differentiating when not being exactly equal. It's recommended to change this comparison for each case but we opted to try with a easily implementable and usable method to serve as a base.

5) *Root Cause Transposal*: Root Cause detection is difficult to obtain in a fully automated process without context-awareness. We split the method to assign a root cause in 2 ways: The first option will be rule-based while the second will use the most similar anomaly to obtain the most probable root cause. The preferential method will always be to use the most similar anomaly to transpose the root cause to the candidate chain if they share a similar structure as it contemplates the similar anomalies. If they are not similar enough or for some reason, we cannot transpose the root, we default to the rule-based approach.

Rules would need to be reviewed in a by case scenario. In our case, the rule is to search the higher tier nodes in the chain subgraph and assign them as the root. We walk the subgraph until we find the node(s) closest to the root of the topology graph and select them as potential root cause locations. If we think back to the most basic examples, most of them are supported in a dependency layout where a failing node propagates its problems to the dependant nodes (e.g., host containing virtual machines, terminals connected to a switch), which is our motivation to default to this rule when we cannot provide a more appropriate diagnostic.

The second method is when we have a similar anomaly available. Starting from the root node(s) of the known anomaly and together with its error description and layer, we apply it to the candidate chain of events. It works particularly well for scenarios rich with similar, recurrent events.

By looking at the diagram 5 we can notice the different color codes being the different error descriptions for each node (nodes of the same color, share the same error description and nodes in bold are identified as root cause). We select the candidate's root cause location by comparing the error description and the root nodes' layer in the anomaly subgraph, transposing its results to the candidate chain.

D. Decision Result

The final stage, named decision, is where we commit the results. Using the resulting maximum similarity value as the confidence in our evaluation, thresholds were defined that establish whether a candidate chain can be trusted or not. If the result is positive, the candidate chain of events can either be stored in the knowledge base as an approved anomaly with the same error ID as its most similar anomaly (if the similarity is high), or it can be added to the knowledge base but with a

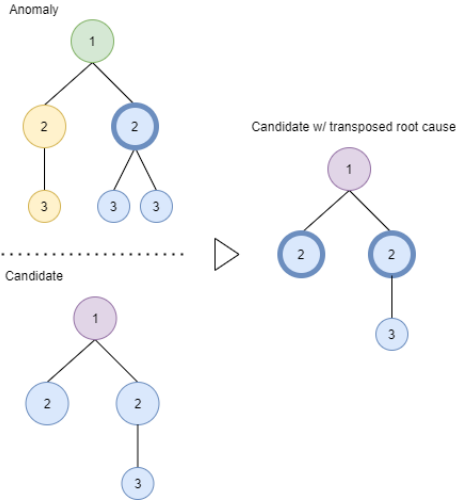


Fig. 5. Root cause using transpose based approach

different error ID (if the similarity is smaller). If the result is negative, it will be written in the temporary database.

Our knowledge base entries are composed of the **events** field, which is a list of the events we deemed as correlated, each containing its origin node, error description, start and end timestamp. We then have the *root* field, a list of root nodes, and the *cluster* where it is declared the error ID.

IV. RESULTS

In this chapter, we will explain the validation process behind our system. For performance comparison, the hardware used to conduct the test had a 6 core CPU (Intel Core i7-9750H) and 16Gb of RAM for reference.

A. Result Validation

The strategy to validate our results will be to use a real world dataset containing roughly 5000 events (already cleaned and filtered from the original untreated version) from a fault management system collecting events from an Asian operator (layout on fig. 6). Unfortunately it is impossible at the time of writing this paper to provide public access to the database. We hope by selecting small samples of chains of events manually selected to be introduced in our knowledge base, we can achieve the designed flow similarly to how it would operate in a real world installation. This knowledge base will be used as the ground truth for the rest of our tests. It was selected a total of 7 correlations.

1) *Correlation Method*: The first step that requires validation is the correlation methods. We selected the key parameters we wish to track during our tests. We introduced the 5000 events, 1000 events per batch. We did not introduce overlapping in this phase, so we can better track the indicators we are collecting.

Using three different static windows, our intention is to collect information from a window we deem is too small (10 seconds), another we find too large (2 hours), and one we consider a good average value (120 seconds) considering the

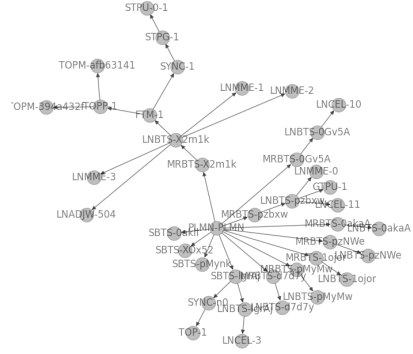


Fig. 6. Network topology on the evaluated dataset

Correlation Methods	Chains of events (count)	Unique anomalies	Error IDs	Unique Anomalies/Error IDs	Unapproved Anomalies	Approved anomalies	Avg. Chains of events size	Avg. Comparison thrust	Stand. Dev. Comparison Thrust	Max. Chain of events size
Static (10s)	6356	19	8	2.37	21	6313	1.176	0.987	0.063	10
Static (120s)	2306	31	14	2.21	67	2304	3.160	0.960	0.093	33
Static (2h)	223	35	18	1.94	32	223	29.411	0.906	0.152	504
Dynamic	990	44	22	2.00	41	990	7.270	0.952	0.098	132
Overlaying	605	37	21	1.76	30	603	11.811	0.959	0.107	1227

TABLE I
COLLECTED INDICATORS FOR EACH CORRELATION METHOD

average distance between each event for each branch in our dataset. Those are considered the extreme cases in which the chains are either cut too short or connecting chains that should be independent. As expected, the other three methods manage to have values in between. An especially useful indicator of the method is to look at the average comparison thrust. From our dataset study, we established that 6% of our errors did not have a specified error description and, as such, should be harder to diagnose. It is then expected that a good average thrust value is around 0.94, which the dynamic window is the closest.

To better distinguish the results between the Overlay and Dynamic window methods, we delved into the specific results, and collected examples (such as the one specified in II) that show the dynamic window is splitting chains that are too large in time. It is also visible in the maximum chain size of both methods. We then determined that many alarms are manually deactivated due to the coincident end timestamps in a large number of events present in the dataset. This leads us to believe that, in our dataset, it is required to use methods that provide more emphasis on the start timestamp.

An interesting remark is the Unique Anomalies to Error IDs ratio, which is similar among all methods. This shows that our ability to compare and clustering process will have a certain flexibility to correlation method changes due to the newly learned chains added to the knowledge base sharing

Chain	Node	Error Type	Start Timestamp	End Timestamp
O1	D1	LNCCEL-11 Cell operation degraded	19.08.30 17:24:17	19.09.05 11:42:36
		LNCCEL-11 Cell operation degraded	19.08.30 22:15:53	19.09.05 11:42:36
	D2	LNBTS-pzbxw Base station notification	19.08.30 22:20:21	19.09.05 11:42:36
D2	LNBTS-pzbxw	Base station connectivity degraded	19.09.04 16:38:12	19.09.05 11:42:36
	MRBTS-pzbxw	NE O&M connection failure	19.09.04 19:44:15	19.09.05 11:42:26

TABLE II
CHAIN OF EVENTS EXAMPLE

characteristics with the ones we will be correlating and chaining at later stages. This is attributed to the high recurrency of anomalies in this type of network. From evaluating the dataset, we conclude that most events are being generated in busy hours, indicating that the higher usage leads to more problems surfacing (refer to fig. 7).

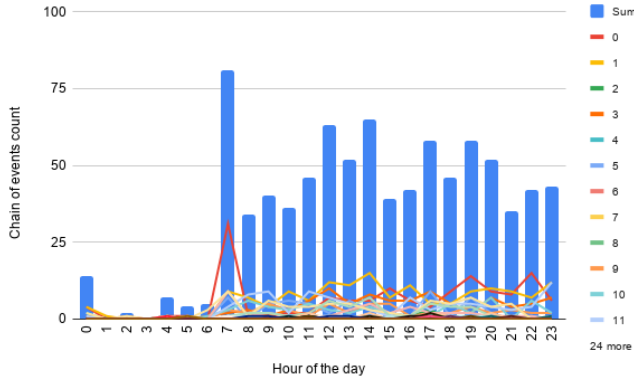


Fig. 7. Anomaly activation per hour of day

It is possible to notice that by varying the number of initial events we include, there is a high variation in the final results (results in table III). This result also denotes the importance of having a correct and broader initial knowledge base to obtain better results.

Initial Chains	Unique	Approved	Unapproved
1	10	332	511
3	24	838	47
7	32	857	39

TABLE III
INITIAL CHAINS COMPARISON

B. Performance Evaluation

Our system's performance is vital to have an idea of the applicability of the system in a real-world application. We also require to evaluate the performance versus results trade-off for each method.

1) *Profiling*: Dissecting the computational power required for each stage of our system will provide us insight into where we are spending the most time processing and hopefully help us concentrate our optimization efforts. We tested the dataset on batches of 1000 with 500 events overlapping (1500 events

each batch) using the dynamic window correlator and the seven initial cases on the knowledge base obtaining the results in table IV.

Events	Chains of events	Cumulative Chains of events	Approved	Error IDs
1000	70	70	68	17
2000	206	276	266	22
3000	199	475	462	23
4000	232	707	693	25
5000	324	1031	989	35

Events	Correlation (s)	Comparison (s)
1000	0.0332	18.2501
2000	0.0303	76.3990
3000	0.0461	281.9351
4000	0.0554	257.2569
5000	0.0184	862.0858

TABLE IV
INITIAL SYSTEM VERSION PERFORMANCE METRICS

It is easily recognizable that most of the computational workload is spent in the comparison phase. By further evaluating, we understand the comparison process's exhaustive nature (where we compare each candidate to every anomaly) may require optimization such as pre-processing before committing to compare. Through the system evolution, it's an exponential increase (refer to fig. 8).

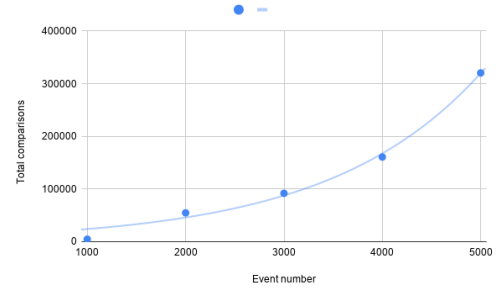


Fig. 8. Comparisons per batch

We identify the need for an optimization process in the comparison in order to reduce the number of comparisons. Even though we can think of multiple strategies, we opted to not include candidate chains in the knowledge base when they are very similar to the one already present, resulting in improved processing times in table V. We consider that if an anomaly is very similar to another one already existing, we will not be adding no information in the knowledge base.

Events	Correlation (s)	Comparison (s)
1000	0.0234259	1.1179527
2000	0.0257114	1.3814406
3000	0.0276691	1.4180092
4000	0.0631363	0.7313271
5000	0.01933679999	3.1856976

TABLE V
FINAL VERSION PERFORMANCE ANALYSIS

2) *Input Requirements*: One of the initial concerns is that the system can evolve and have fewer cases requiring external evaluation to arrive at an almost zero touch goal. To test this, we introduced a random chance to accept cases that would otherwise be sent to the temporary database simulating a human operator evaluation. It is not ideal as the quality of the accepted chains is random, but the system's parameters growth can give us a good insight.

Anomaly DB Start	Percentage added	Batch 1	Batch 2	Batch 3	Batch 4	Batch 5
1	0	119	127	83	51	131
1	1%	36	1	7	6	21
1	5%	18	2	13	5	22
3	0	7	1	7	4	28
3	1%	7	1	7	4	24
3	5%	7	0	10	4	10
7	0	2	1	12	3	25
7	1%	1	1	6	4	23
7	5%	1	1	10	0	20

TABLE VI
INPUT REQUIREMENTS PER STAGE

We can see in table VI from the unapproved chains for each batch that as we increase the number of approved chains present in the knowledge base, the number of pending cases quickly decreases. We are left with chains present in some batches (especially the third and fifth) that are outliers, and our system will have trouble analyzing.

3) *Comparison Methods*: Another portion that requires our attention are the comparison methods. To evaluate them, without testing all combinations of our four comparison methods, we will rely on the principle that each method's impact is equal to its negative impact when not in use. We established a baseline with all four and then tested without each at a time. Each method is recognized by its first letter.

Comparison methods	Unique Anomalies	Error IDs	Anomalies	Unapproved
Full	44	21	988	43
S/H/E	45	25	1000	31
S/H/B	40	18	852	179
S/E/B	13	8	829	202
H/E/B	57	30	853	178

Comparison methods	Average Thrust	Standard Deviation	Comparison Time	Impact (Delta %)	Performance lift
Full	0.9517	0.0992	12.1577	0	0
S/H/E	0.9556	0.0776	11.2279	0.4	108.3
S/H/B	0.8943	0.1715	12.1750	6.1	92.2
S/E/B	0.9222	0.1362	3.3978	2.8	358.3
H/E/B	0.8986	0.1478	5.6169	2.4	60.5

TABLE VII
COMPARISON METHODS ANALYSIS

From the results depicted in table VII, we can take a few conclusions. We understand that the branching comparison is the least impactful in our scenario and that the error description is fundamental due to the lack of topological complexity in our case study. The error description helps to have some context awareness in the comparison. The performance increase of removing the branching comparison is small, so we advise running it even if with less weight in the final result. We can also conclude how important the Subgraph, Error Description

and Hierarchy comparison are to the obtained results through the average thrust.

V. CONCLUSION

The initial problem we were set to resolve was a system capable of providing RCA on a telecom network scenario. We had individual events collected by a FMS, and we were required to identify chains among them according to correlation rules, while identifying where the root cause is. We tried to create a modular system that can easily be modified and adapted to specific cases a user may have while also providing methods of our own that encompass a general approach to the design. We introduced multiple correlating rules as well as different comparison methods and output decisions. Our system is capable of identifying anomalies in systems based on a known issues list where those known issues are either part of the network's preestablished error traces or past recorded occurrences. We then compare the current system behavior to verify whether we have repeating patterns to the known issues that may be currently re-surfacing. This task is currently handled manually, meaning that it takes a long time to complete, and requires domain experts to do it. Our system replaces a part of the workload attributed to humans while reducing the time required to diagnose the most recurrent problems this type of network face, which by our analysis, are a significant percentage of the daily anomalies occurring. We also allow the option to flag occurrences, we are less confident in providing our RCA to be tracked and evaluated by those human operators. There is also the possibility to create new workflows for untrusted analysis to be reevaluated.

We are confident that not only have we developed a system capable of answering our initial problem. We believe we created a solid building ground to keep developing on this topic moving forward since we expect it will be even more required in the near future. Different scenarios should be tested to evaluate the current state of the system while better correlation, comparison, and root cause identification methods can and should be researched, possibly using our system as the foundation. We hope that we can reach a stage where we completely remove human interaction from the workflow loop. The need for automation will keep rising, and we just added a new option to the multitude of available tools.

While we have addressed the problem and achieved initial results, there are several aspects we intend to address in the future work. An important aspect is the capability to run this system in multiple network types (not only hierarchical) and network segments (or domains), possibly with definable cross-segment analysis required by technologies like network slicing being more and more prominent [13]. Another research direction will be to evaluate new root cause identification strategies, possibly scenario-focused and less general than the ones we developed. It's hard to tackle an RCA problem without a specific context in mind where results can be improved by having a limited and fine-tuned approach to each area of application. We also experimented with a few different features that we dropped due to the lack of clear evidence

of their value and correctness with our dataset. We tried to feed the system with different types of graphs related to these systems instead of only using topology. We called this feature *Logic Graphs* as the graphs we generated were according to specific rules (e.g., using the branch types). We predict this type of feature will open new perspectives to the RCA process leading to better diagnosis. The other feature we developed is called *Hidden Connections* where we analyze our dataset statistically to predict possible connections in the topology that are not represented explicitly. This feature was motivated by the "handover problem" that happens in telecom networks. Geographically close antennas have logic connections to deal with the handover that may transport correlated alarms through branches. We achieve this by analyzing the number of times events happen in a specific order (explained in fig. 9).

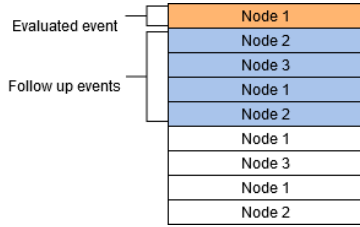


Fig. 9. Frequency Matrix processing

These methods should be further investigated and tested as we consider they seem promising on different scenarios than the one we tested.

ACKNOWLEDGMENT

This project was partially funded under project 5GAIner, a 5G+AI joint university-industry initiative in Aveiro, and by FCT/MCTES through national funds and when applicable co-funded EU funds under the project UIDB/50008/2020-UIDP/50008/2020.

REFERENCES

- [1] Nokia, *Future X Network cost economics*, 2019, CID202288.
- [2] Vaishnavi, Ishan and Ciavaglia, Laurent, *Challenges towards automation of live telco network management: Closed control loops*, 16th CNSM, 2020.
- [3] A. Boubendir, F. Guillemin, S. Kerboeuf, B. Orlandi, F. Faucheux and J. Lafrayette, *Network Slice Life-Cycle Management Towards Automation*, 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Arlington, USA, pp. 709-711, 2019.
- [4] Xie, Min and Zhang, Qiong and Gonzalez, Andres J. and Gronsund, Pal and Palacharla, Paparao and Ikeuchi, Tadashi, *Service Assurance in 5G Networks: A Study of Joint Monitoring and Analytics*, IEEE PIMRC, 2019.
- [5] Pablo Munoz and Isabel De La Bandera and Emil J. Khatib and Ana Gomez-Andrades and Inmaculada Serrano and Raquel Barco, *Root Cause Analysis Based on Temporal Analysis of Metrics Toward Self-Organizing 5G Networks*, IEEE Transactions on Vehicular Technology, 2017.
- [6] Juan Qiu and Qingfeng Du and Shuang Li Zhang and Chongshu Qian, *A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications*, Applied Sciences (Switzerland), 2020.
- [7] Álvaro Brandón and Marc Solé and Alberto Huélamo and David Solans and María S. Pérez and Victor Muntés-Mulero, *Graph-based root cause analysis for service-oriented and microservice architectures*, Journal of Systems and Software, 2020.

- [8] Marc Solé and Victor Muntés-Mulero and Annie Ibrahim Rana and Giovanni Estrada, *Survey on Models and Techniques for Root-Cause Analysis*, CoRR, 2017.
- [9] D. Mulvey, C. H. Foh, M. A. Imran and R. Tafazolli, *Cell Fault Management Using Machine Learning Techniques*, IEEE Access, vol. 7, 2019.
- [10] K. Yang, R. Liu, Y. Sun, J. Yang and X. Chen, *Deep Network Analyzer (DNA): A Big Data Analytics Platform for Cellular Networks*, IEEE Internet of Things Journal, vol. 4, no. 6, pp. 2019-2027, 2017.
- [11] X. Nie, Y. Zhao, K. Sui, D. Pei, Y. Chen and X. Qu, *Mining causality graph for automatic web-based service diagnosis*, 2016 IEEE 35th International Performance Computing and Communications Conference (IPCCC), 2016.
- [12] R. Harper and P. Tee, *A Method for Temporal Event Correlation*, 2019 IFIP/IEEE Symposium on Integrated Network and Service Management (IM), Arlington, USA, pp. 13-18, 2019.
- [13] X. Li et al., *Network Slicing for 5G: Challenges and Opportunities*, IEEE Internet Computing, vol. 21, no. 5, pp. 20-27, 2017.